

member of the list L.

```
$ maxima -q
(%i1) L: create_list(i * x, x, [a, b, c], i, [1, 2, 3, 4]);
(%o1) [a, 2 a, 3 a, 4 a, b, 2 b, 3 b, 4 b, c, 2 c, 3 c, 4 c]
(%i2) first(L);
(%o2) a
(%i3) seventh(L);
(%o3) 3 b
(%i4) last(L);
(%o4) 4 c
(%i5) third(L); last(L);
(%o5) 3 a
(%o6) 4 c
(%i7) L;
(%o7) [a, 2 a, 3 a, 4 a, b, 2 b, 3 b, 4 b, c, 2 c, 3 c, 4 c]
(%i8) quit();
```

Again, note that the list L is still not modified. However, we may need to modify the existing lists, and none of the above functions will do that. It could be achieved by assigning the return values of the various list recreation functions back to the original list. However, there are a few functions, which do modify the list right away.

List manipulations

The following are the two list manipulating functions provided by Maxima:

- `push(v, L)` - inserts 'v' at the beginning of the list L.
- `pop(L)` - removes and returns the first element from list L. L must be a symbol bound to a list, not the list itself, in both the above functions, for them to modify it. Also, these functionalities are not available by default, so we need to load the basic Maxima file. Check out the demonstration below.

We may display L after doing these operations, or even check the length of L to verify the actual modification of L. In case we need to preserve a copy of the list, the function `copylist()` can be used.

```
$ maxima -q
(%i1) L: makelist(2 * x, x, 1, 10);
(%o1) [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
(%i2) push(0, L); /* This doesn't work */
(%o2) push(0, [2, 4, 6, 8, 10, 12, 14, 16, 18, 20])
(%i3) pop(L); /* Nor does this work */
(%o3) pop([2, 4, 6, 8, 10, 12, 14, 16, 18, 20])
(%i4) load(basic); /* Loading the basic Maxima file */
(%o4) /usr/share/maxima/5.24.0/share/macro/basic.mac
(%i5) push(0, L); /* Now, this works */
(%o5) [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
(%i6) L;
(%o6) [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
(%i7) pop(L); /* Even this works */
(%o7) 0
(%i8) L;
```

```
(%o8) [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
(%i9) K: copylist(L);
(%o9) [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
(%i10) length(L);
(%o10) 10
(%i11) pop(L);
(%o11) 2
(%i12) length(L);
(%o12) 9
(%i13) K;
(%o13) [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
(%i14) L;
(%o14) [4, 6, 8, 10, 12, 14, 16, 18, 20]
(%i15) pop([1, 2, 3]); /* Actual list is not allowed */
arg must be a symbol [1, 2, 3]
#0: symbolcheck(x=[1,2,3])(basic.mac line 22)
#1: pop(l=[1,2,3])(basic.mac line 26)
-- an error. To debug this try: debugmode(true);
(%i16) quit();
```

Advanced list operations

And finally, here is a bonus of two sophisticated list operations:

- `sublist_indices(L, p)` - returns the list indices for the members of the list L, for which predicate 'p' is 'true'.
- `assoc(k, L, d)` - L must have all its members in the form of `x op y`, where `op` is some binary operator. Then, `assoc()` searches for 'k' in the left operand of the members of L. If found, it returns the corresponding right operand, otherwise it returns 'd'; or it returns false, if 'd' is missing.

Check out the demonstration below for both the above operations

```
$ maxima -q
(%i1) sublist_indices([12, 23, 57, 37, 64, 67], primep);
(%o1) [2, 4, 6]
(%i2) sublist_indices([12, 23, 57, 37, 64, 67], evenp);
(%o2) [1, 5]
(%i3) sublist_indices([12, 23, 57, 37, 64, 67], oddp);
(%o3) [2, 3, 4, 6]
(%i4) sublist_indices([2 > 0, -2 > 0, 1 = 1, x = y], identity);
(%o4) [1, 3]
(%i5) assoc(2, [2^r, x+y, 2=4, 5/6]);
(%o5) r
(%i6) assoc(6, [2^r, x+y, 2=4, 5/6]);
(%o6) false
(%i7) assoc(6, [2^r, x+y, 2=4, 5/6], na);
(%o7) na
(%i8) quit();
```



By: Anil Kumar Pugalia

The author is a gold medalist from NIT, Warangal and IISc, Bengaluru. Mathematics and knowledge-sharing are two of his many passions. Learn more about him at <http://splays.in>. He can be reached at email@sarika-pugs.com.